

```
from google.colab import files
```

```
uploaded = files.upload()
```

[Choose Files](#) Housing.csv

Housing.csv(text/csv) - 29981 bytes, last modified: 6/20/2026 - 100% done

Saving Housing.csv to Housing (1).csv

```
import pandas as pd
```

```
df = pd.read_csv("Housing.csv")
```

```
df.head()
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement
0	13300000	7420	4	2	3	yes	no	no
1	12250000	8960	4	4	4	yes	no	no
2	12250000	9960	3	2	2	yes	no	yes
3	12215000	7500	4	2	2	yes	no	yes
4	11410000	7420	4	1	2	yes	yes	yes

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

```
print(df.shape)
print(df.columns)
```

```
(545, 13)
Index(['price', 'area', 'bedrooms', 'bathrooms', 'stories', 'mainroad',
       'guestroom', 'basement', 'hotwaterheating', 'airconditioning',
       'parking', 'prefarea', 'furnishingstatus'],
      dtype='object')
```

House Price Prediction

Task 1: Data Loading and Exploration



```
print(df.isnull().sum())
```

```
price      0
area       0
bedrooms   0
bathrooms  0
stories    0
mainroad   0
```

```

guestroom      0
basement       0
hotwaterheating 0
airconditioning 0
parking        0
prefarea       0
furnishingstatus 0
dtype: int64

```

```
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 545 entries, 0 to 544
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   price                 545 non-null   int64
1   area                 545 non-null   int64
2   bedrooms             545 non-null   int64
3   bathrooms            545 non-null   int64
4   stories              545 non-null   int64
5   mainroad             545 non-null   object
6   guestroom            545 non-null   object
7   basement             545 non-null   object
8   hotwaterheating      545 non-null   object
9   airconditioning      545 non-null   object
10  parking              545 non-null   int64
11  prefarea             545 non-null   object
12  furnishingstatus     545 non-null   object
dtypes: int64(6), object(7)
memory usage: 55.5+ KB

```

```
df.drop_duplicates(inplace=True)
```

```
print("New Shape:", df.shape)
```

```
New Shape: (545, 13)
```

```
df = pd.get_dummies(df, drop_first=True)
```

```
df.head()
```

	price	area	bedrooms	bathrooms	stories	parking	mainroad_yes	guest
0	13300000	7420	4	2	3	2	True	
1	12250000	8960	4	4	4	3	True	
2	12250000	9960	3	2	2	2	True	
3	12215000	7500	4	2	2	3	True	
4	11410000	7420	4	1	2	2	True	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
print(df.shape)
```

```
(545, 14)
```

```
from sklearn.model_selection import train_test_split

X = df.drop("price", axis=1)
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print(X_train.shape)
print(X_test.shape)
```

```
(436, 13)
```

```
(109, 13)
```

```
from sklearn.linear_model import LinearRegression

lr = LinearRegression()

lr.fit(X_train, y_train)

lr_pred = lr.predict(X_test)
```

```
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

import numpy as np

mae_lr = mean_absolute_error(y_test, lr_pred)

rmse_lr = np.sqrt(mean_squared_error(y_test, lr_pred))

r2_lr = r2_score(y_test, lr_pred)

print("MAE:", mae_lr)
print("RMSE:", rmse_lr)
print("R2:", r2_lr)
```

```
MAE: 970043.4039201636
RMSE: 1324506.9600914386
R2: 0.6529242642153184
```

```
print(df.isnull().sum())
```

```
price          0
area           0
bedrooms       0
bathrooms      0
stories        0
parking        0
mainroad_yes   0
guestroom_yes  0
basement_yes   0
hotwaterheating_yes  0
airconditioning_yes  0
prefarea_yes   0
furnishingstatus_semi-furnished  0
furnishingstatus_unfurnished  0
dtype: int64
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 545 entries, 0 to 544
Data columns (total 14 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   price                                     545 non-null    int64
1   area                                     545 non-null    int64
2   bedrooms                                545 non-null    int64
3   bathrooms                               545 non-null    int64
4   stories                                 545 non-null    int64
5   parking                                 545 non-null    int64
6   mainroad_yes                            545 non-null    bool
7   guestroom_yes                           545 non-null    bool
8   basement_yes                            545 non-null    bool
9   hotwaterheating_yes                     545 non-null    bool
10  airconditioning_yes                      545 non-null    bool
11  prefarea_yes                             545 non-null    bool
12  furnishingstatus_semi-furnished          545 non-null    bool
13  furnishingstatus_unfurnished             545 non-null    bool
dtypes: bool(8), int64(6)
memory usage: 29.9 KB
```

✓ Task 3: Model Building

```
from sklearn.model_selection import train_test_split

X = df.drop("price", axis=1)
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

```
print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (436, 13)
Testing Data: (109, 13)
```

```
from sklearn.linear_model import LinearRegression

lr = LinearRegression()

lr.fit(X_train, y_train)

lr_pred = lr.predict(X_test)
```

```
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
import numpy as np

mae_lr = mean_absolute_error(y_test, lr_pred)

rmse_lr = np.sqrt(mean_squared_error(y_test, lr_pred))

r2_lr = r2_score(y_test, lr_pred)

print("Linear Regression Results")
print("MAE =", mae_lr)
print("RMSE =", rmse_lr)
print("R² =", r2_lr)
```

```
Linear Regression Results
MAE = 970043.4039201636
RMSE = 1324506.9600914386
R² = 0.6529242642153184
```

```
from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

rf.fit(X_train, y_train)

rf_pred = rf.predict(X_test)
```

```
mae_rf = mean_absolute_error(y_test, rf_pred)

rmse_rf = np.sqrt(mean_squared_error(y_test, rf_pred))

r2_rf = r2_score(y_test, rf_pred)
```

```
print("Random Forest Results")
print("MAE =", mae_rf)
print("RMSE =", rmse_rf)
print("R² =", r2_rf)
```

```
Random Forest Results
MAE = 1021546.0353211008
RMSE = 1400565.9728553821
R² = 0.611918531405699
```

```
comparison = pd.DataFrame({
    "Model": ["Linear Regression", "Random Forest"],
    "MAE": [mae_lr, mae_rf],
    "RMSE": [rmse_lr, rmse_rf],
    "R2 Score": [r2_lr, r2_rf]
})
```

```
comparison
```

	Model	MAE	RMSE	R2 Score	
0	Linear Regression	9.700434e+05	1.324507e+06	0.652924	
1	Random Forest	1.021546e+06	1.400566e+06	0.611919	

Next steps:

[Generate code with comparison](#)
[New interactive sheet](#)

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
```

```
import os

os.makedirs("charts", exist_ok=True)
```

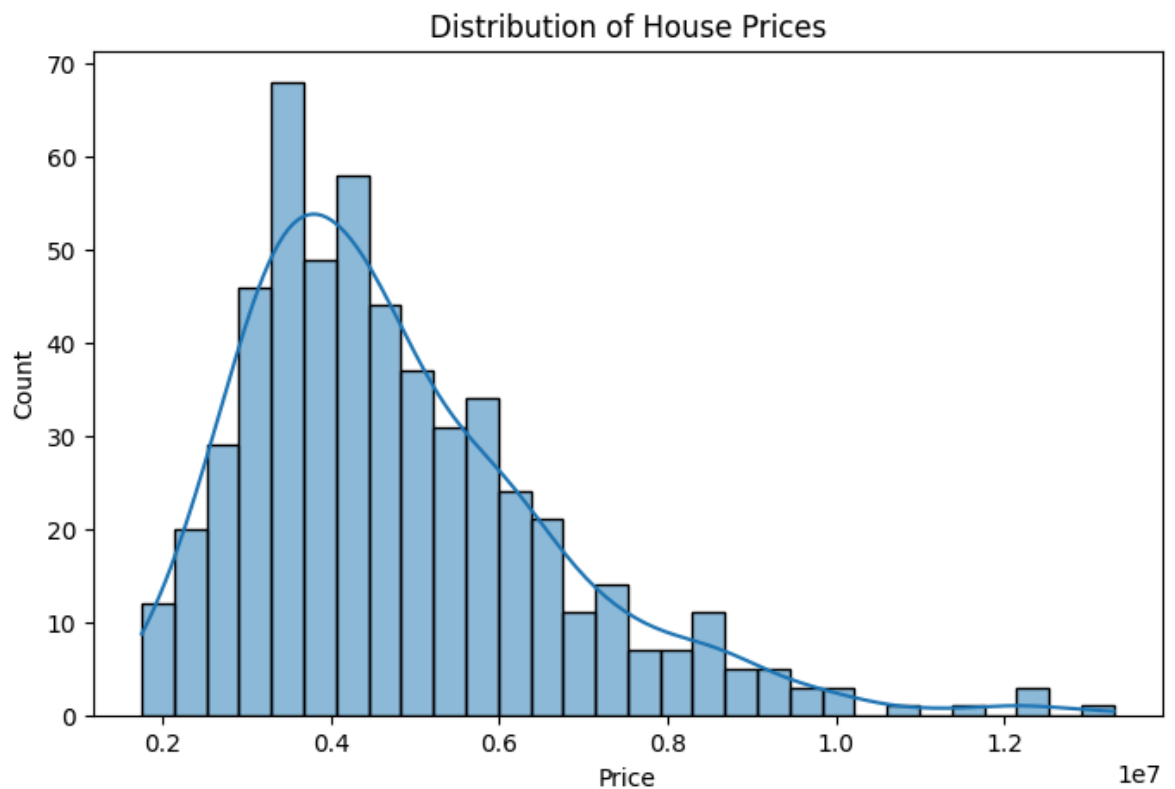
```
plt.figure(figsize=(8,5))

sns.histplot(df["price"], bins=30, kde=True)

plt.title("Distribution of House Prices")
plt.xlabel("Price")
```

```
plt.savefig("charts/price_distribution.png")
```

```
plt.show()
```



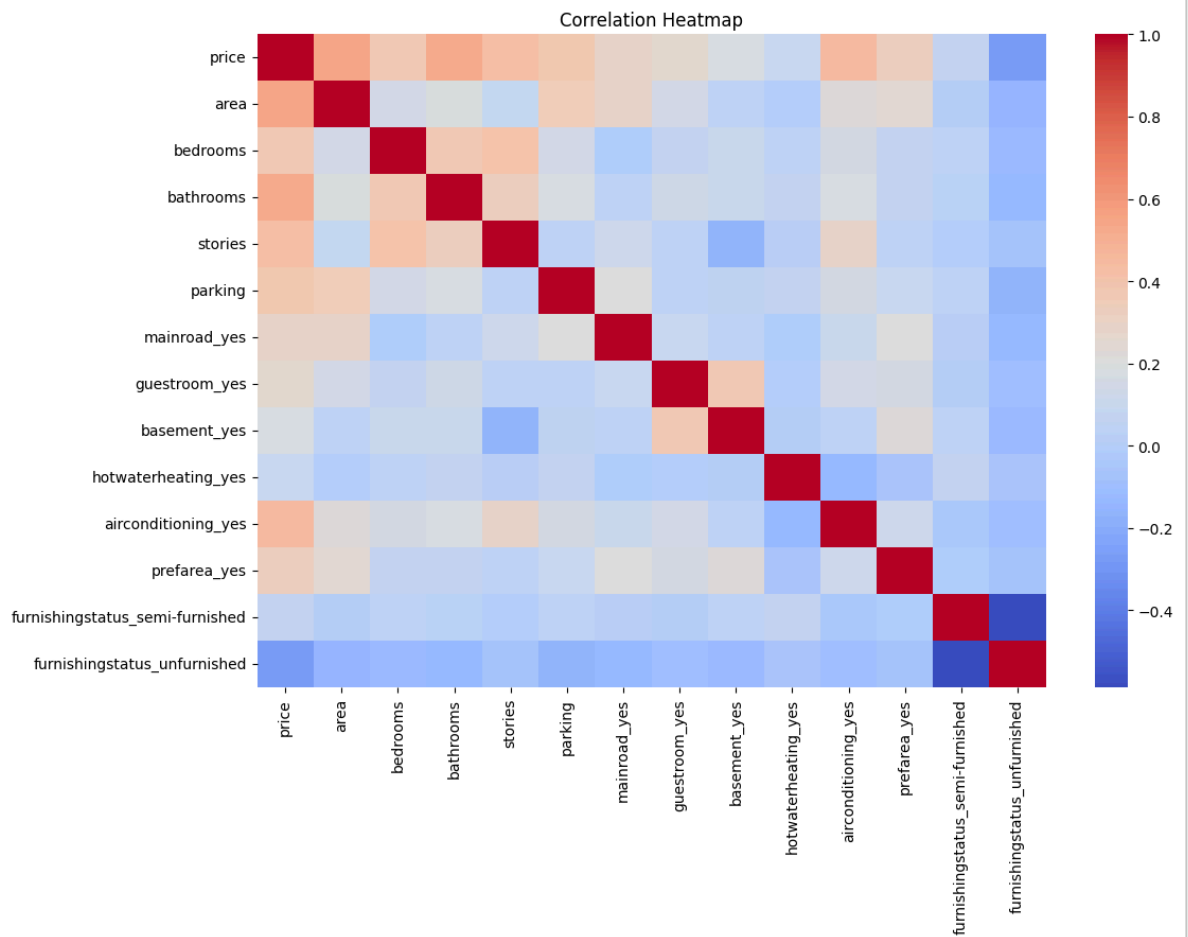
```
plt.figure(figsize=(12,8))
```

```
sns.heatmap(  
    df.corr(),  
    annot=False,  
    cmap="coolwarm"  
)
```

```
plt.title("Correlation Heatmap")
```

```
plt.savefig("charts/correlation_heatmap.png")
```

```
plt.show()
```



```
plt.figure(figsize=(8,6))
```

```
plt.scatter(y_test, lr_pred)
```

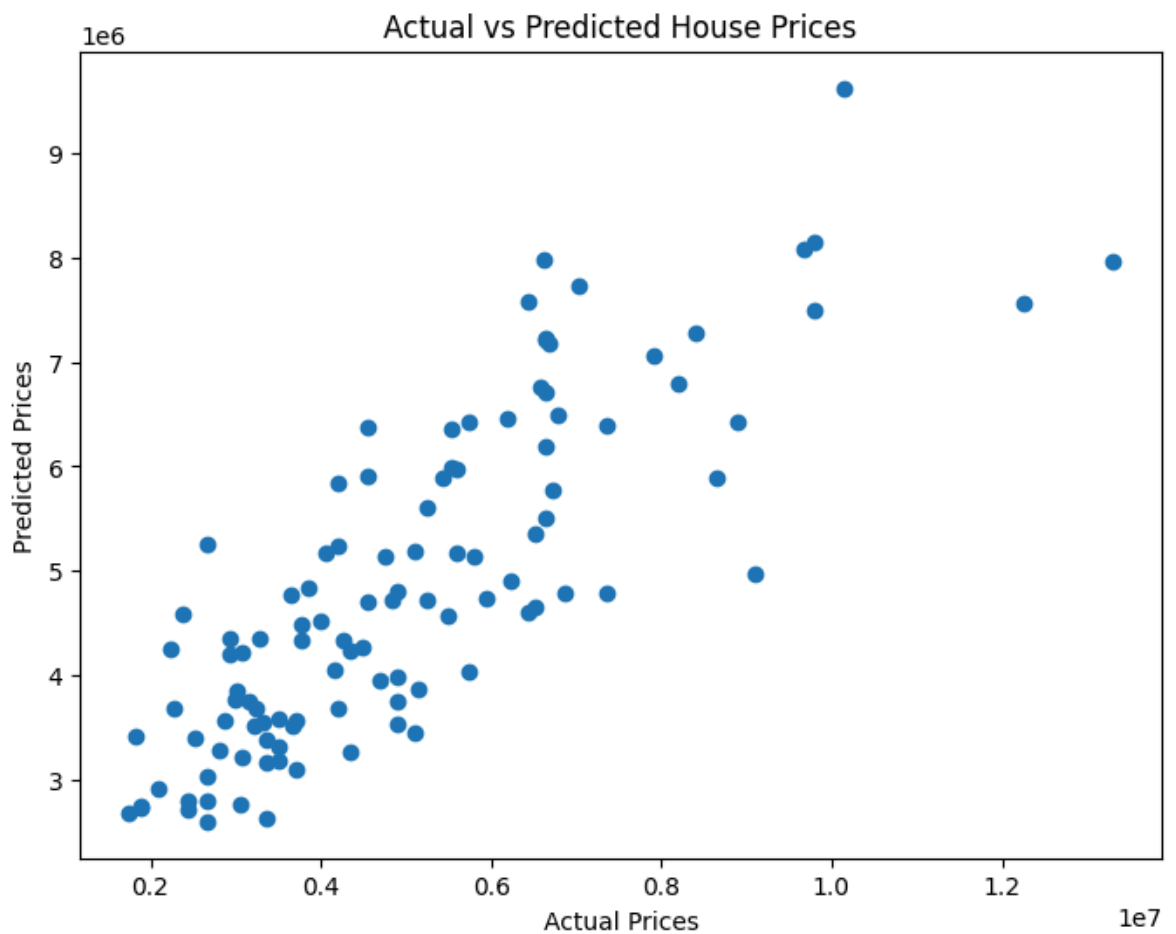
```
plt.xlabel("Actual Prices")
```

```
plt.ylabel("Predicted Prices")
```

```
plt.title("Actual vs Predicted House Prices")

plt.savefig("charts/actual_vs_predicted.png")

plt.show()
```



```
importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": rf.feature_importances_
})

importance = importance.sort_values(
    by="Importance",
    ascending=False
)

importance.head(10)
```

	Feature	Importance	
0	area	0.468430	
2	bathrooms	0.151483	
9	airconditioning_yes	0.062672	
4	parking	0.057455	
3	stories	0.057133	
1	bedrooms	0.048413	
12	furnishingstatus_unfurnished	0.035011	
7	basement_yes	0.030823	
10	prefarea_yes	0.030689	
8	hotwaterheating_yes	0.017226	

Next steps:

[Generate code with importance](#)[New interactive sheet](#)

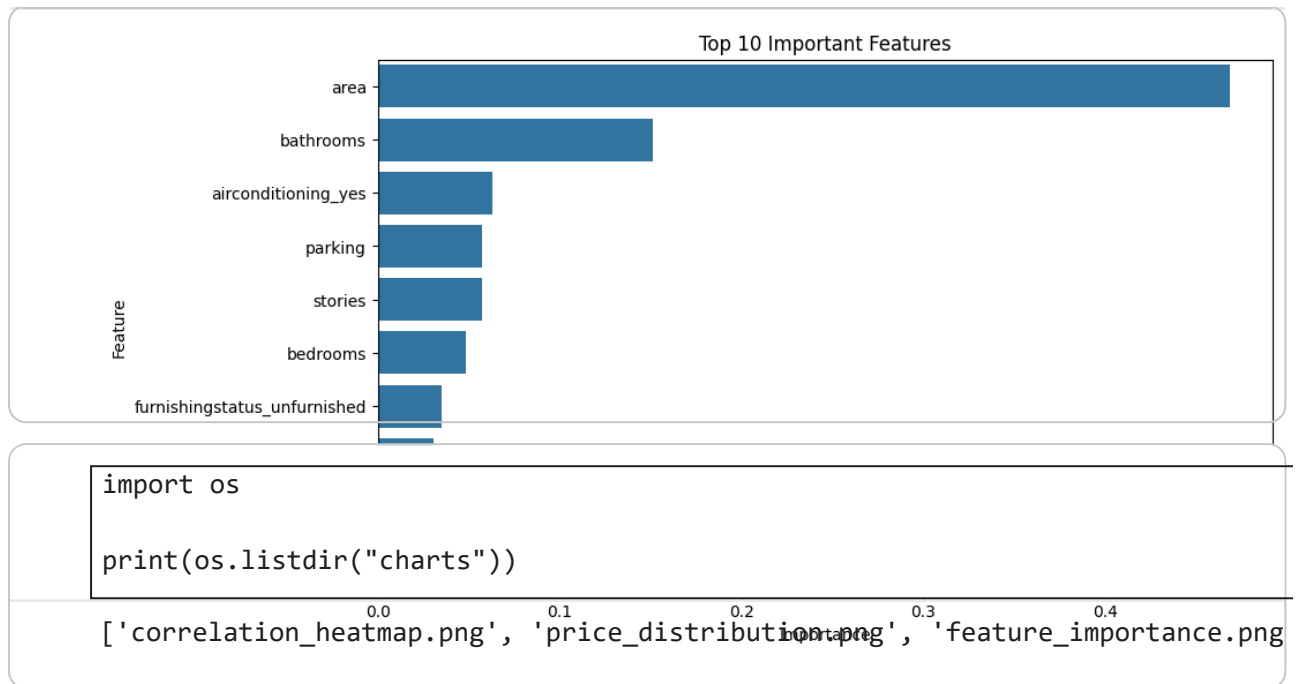
```
plt.figure(figsize=(10,6))

sns.barplot(
    x="Importance",
    y="Feature",
    data=importance.head(10)
)

plt.title("Top 10 Important Features")

plt.savefig("charts/feature_importance.png")

plt.show()
```



✓ Insights and Summary

This project aimed to predict house prices using machine learning techniques. After cleaning and preprocessing the housing dataset, two regression models were trained: Linear Regression and Random Forest Regression. The Linear Regression model performed better, achieving an R^2 score of approximately 0.653, which means it explained about 65% of the variation in house prices. The most influential features affecting house prices were area, bathrooms, air conditioning, parking availability, number of stories, and bedrooms. An interesting observation was that a simple Linear Regression model outperformed the more complex Random Forest model on this dataset. Based on these findings, real estate businesses should pay special attention to property size and key amenities when estimating property values and designing marketing strategies.

Start coding or [generate](#) with AI.